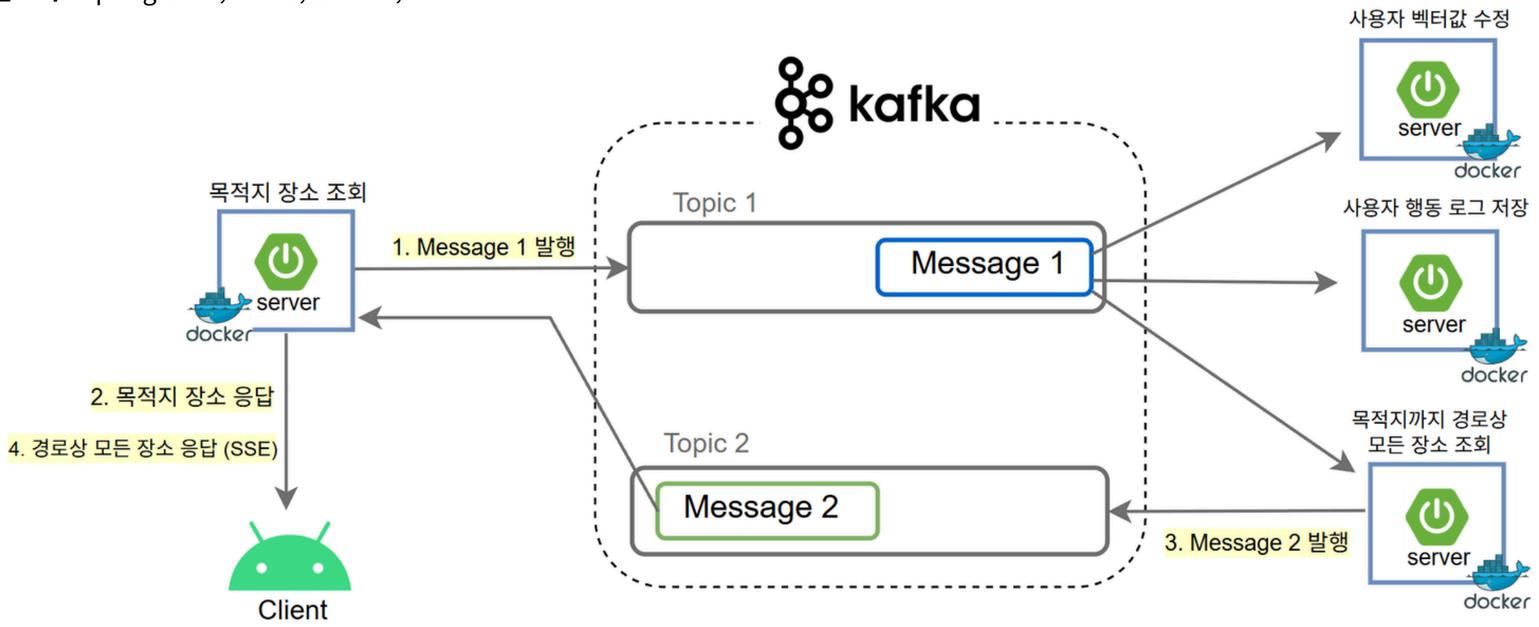
1. Kafka 기반 비동기 처리로 응답 지연 개선 및 부하테스트·모니터링

기술스택 : Spring Boot, Kafka, Docker, K6



[문제 및 원인]

- 성능 저하가 발생한 기능 : 가이드 시작하기 API
- 요청-응답 흐름 안에서 4가지 작업이 묶여있는 구조이고, 이 중 특히 느린 대용량 데이터 조회 작업의 지연이 사용자 응답까지 병목이 전파되어 성능을 저하시킴

[해결 과정]

- 가이드 시작하기 API 특징 : 사용자가 목적지까지 이동 중 경로에 있는 장소에 실제로 근접(GPS로 파악)했을때 해당 장소에 대한 정보 제공
  - 사용자가 장소에 실제로 근접했을때 그 장소에 대한 데이터가 필요한것이므로 즉시 응답하지 않고 후속처리해도 된다고 판단
  - 나머지 작업인 벡터값 수정 및 로그 저장도 즉시 처리하지 않아도 되는 작업이라고 판단 (추천 AI 기능을 위한 작업들)
- 사용자에게 즉시 응답이 필요하지 않은 작업들을 요청-응답 흐름에서 분리하고 비동기 처리하여 병목 전파를 막음

[성과]

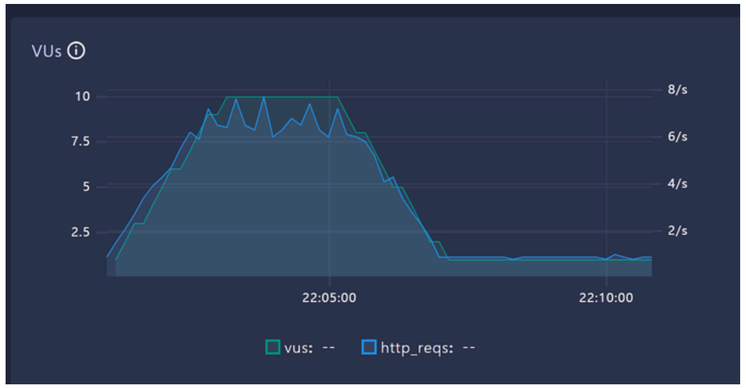
- 느린 작업을 요청-응답 흐름에서 분리해 병목 전파를 막음으로써 응답 지연이 90% 이상 감소
- 특정 작업의 병목 및 장애가 발생해도 전파되지 않는 시스템 결합도가 낮아진 구조로 개선

[부하테스트 및 모니터링]

- 부하테스트 시나리오
  - 일시적으로 많은 사용자가 서비스에 몰려 기능 호출이 2분간 점점 급증한다고 가정
- 부하테스트 스크립트 코드
  - 2분간 VUs를 최대 10까지 점진적으로 증가 → 2분간 유지 → 2분간 1 VU까지 감소 → 4분간 유지

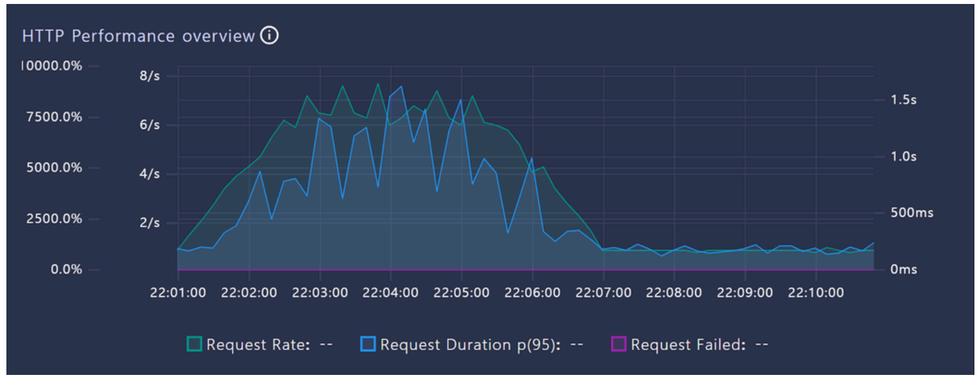
[성능 개선 이전 부하테스트 모니터링]

1. 처리량 측정



→ VUs를 10까지 증가시켜도 처리량이 약 7 TPS 수준에서 정체

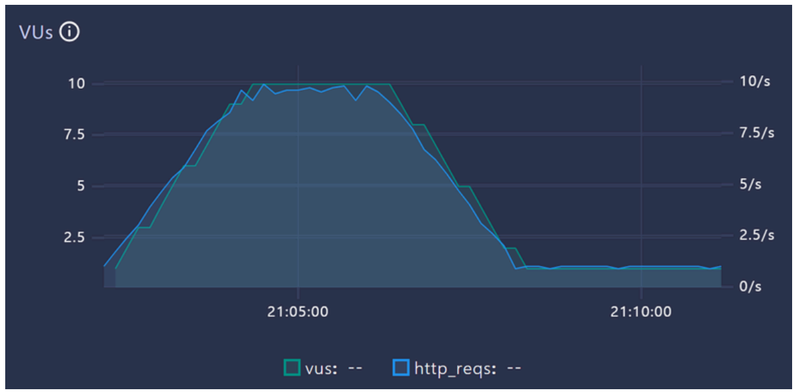
2. 지연 시간 측정



→ 피크 시간대 평균 응답시간이 약 1.5s 까지 증가

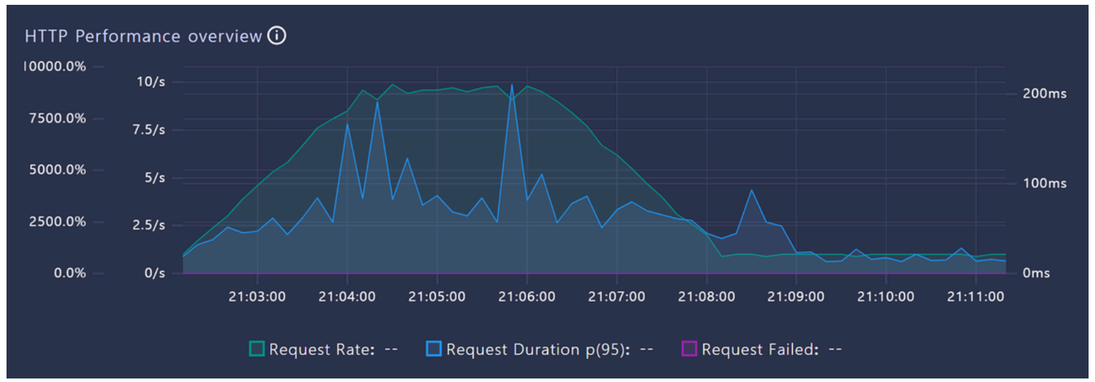
[성능 개선 이후 부하테스트 모니터링]

1. 처리량 측정



→ 10 VUs에서도 10 TPS의 처리량을 유지

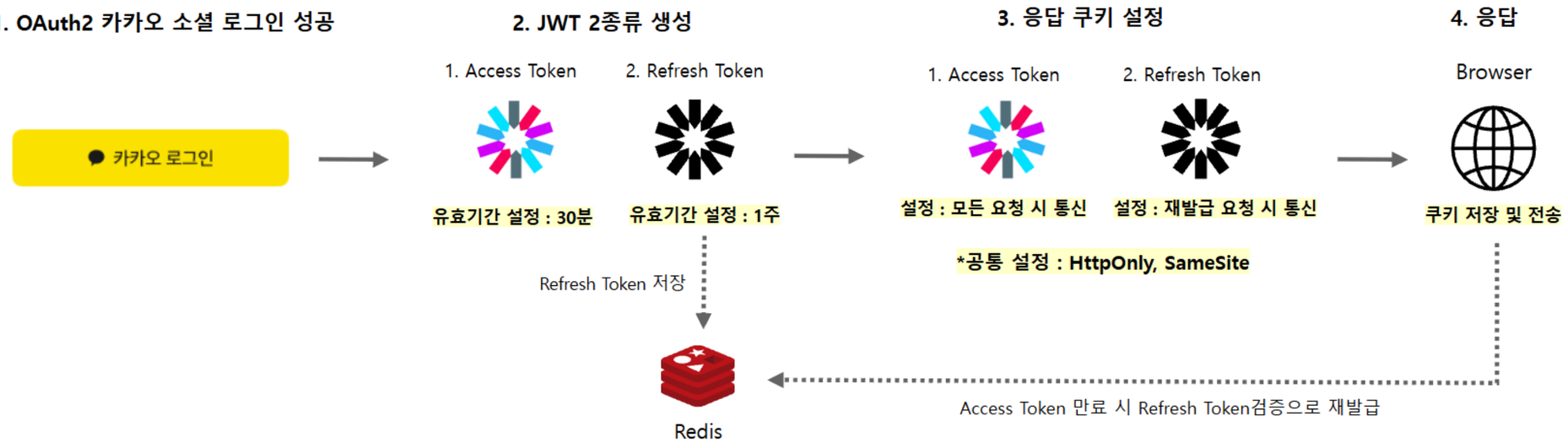
2. 지연 시간 측정



→ 피크 시간이더라도 100ms ~ 200ms (측정 상 약 90% 개선)

2. JWT 인증 구조의 보안성과 편의성 개선

기술스택 : Spring Boot, Spring Security, Redis, OAuth2, JWT



[문제 및 원인]

- 클라이언트의 로컬 스토리지에 JWT(RFC 7519) 인증 토큰을 저장하게 구현하여 기능이 동작은 하지만 분석 결과, 여러 문제가 발생 가능함
  - XSS 발생 시 Local Storage의 인증 토큰이 탈취될 위험
  - 토큰 유효기간이 길면 탈취 시 피해가 커지고, 짧으면 잦은 재로그인으로 사용자 편의성 저하
  - 프론트엔드에서 토큰 저장·전송 로직을 직접 관리해야 하여 구현 복잡도가 증가

[해결 과정]

- HttpOnly Cookie(RFC 6265) 기반 Refresh Token 인증 체제로 전환
  - Access Token은 매 요청 시 서버 앞단에서 서명을 검증하는 출입증 역할로 운용
  - Refresh Token은 Access Token 만료 시 사용자를 다시 확인해 토큰을 재발급받기 위한 재발급 증표 역할로 운용

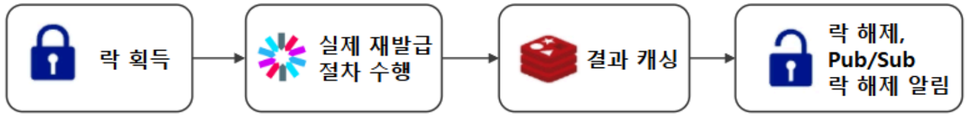
[성과]

- 쿠키에 HttpOnly와 SameSite 속성을 적용해 XSS를 통한 토큰 탈취 위험과 CSRF 공격 위험을 완화
- 짧은 Access Token 유효기간으로 인해 탈취 시 피해 범위 최소화, Refresh Token 기반 자동 재발급 메커니즘을 통해 재로그인 없이 로그인 상태를 지속적으로 유지할 수 있게 하여 사용자 편의성 증대
- 서버 중심의 토큰 관리와 브라우저의 쿠키 자동 전송을 통해 프론트엔드의 토큰 처리 로직을 단순화

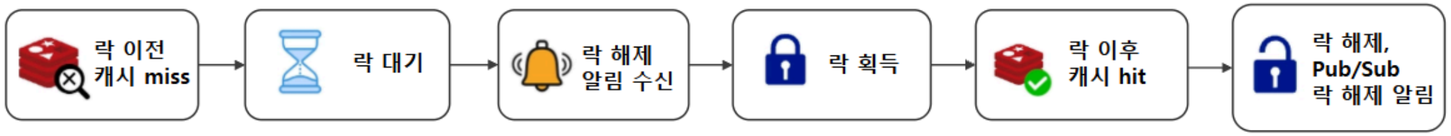
3. JWT 재발급 시 Race Condition을 Redis 분산락으로 제어

기술스택 : Spring Boot, Redis

최초 락 획득 쓰레드



후속 쓰레드 - Case 1



후속 쓰레드 - Case 2



[문제 및 원인]

- 앞선 JWT 인증 구조의 보안성과 편의성을 개선한 후 발생한 추가 문제
- 멀티쓰레드 환경으로 인해 여러 컴포넌트에서 동시에 토큰 만료로 재발급 API를 호출하면 Refresh Token 검증 과정에서 Race Condition이 발생하여 정합성 문제로 재발급 실패

[해결 과정]

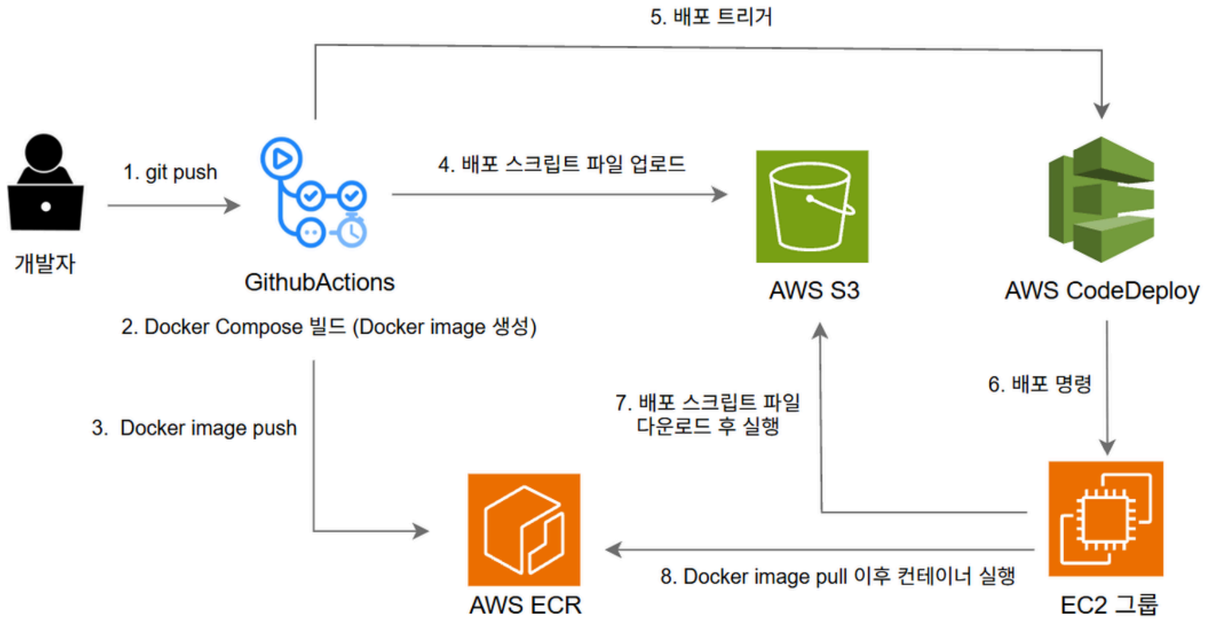
- Redis의 Redisson 분산락을 활용한 Double Checked Locking 패턴
- 실제 재발급은 최초 락 획득 쓰레드만 수행하고, 나머지 후속 쓰레드들은 락 획득 전/후 캐시된 재발급 결과를 확인해 중복 재발급을 방지
- Redisson의 락 설정을 통해 락 대기 시간을 제한하고, 락 자동 해제로 데드락 및 무한 대기 위험을 방지

[성과]

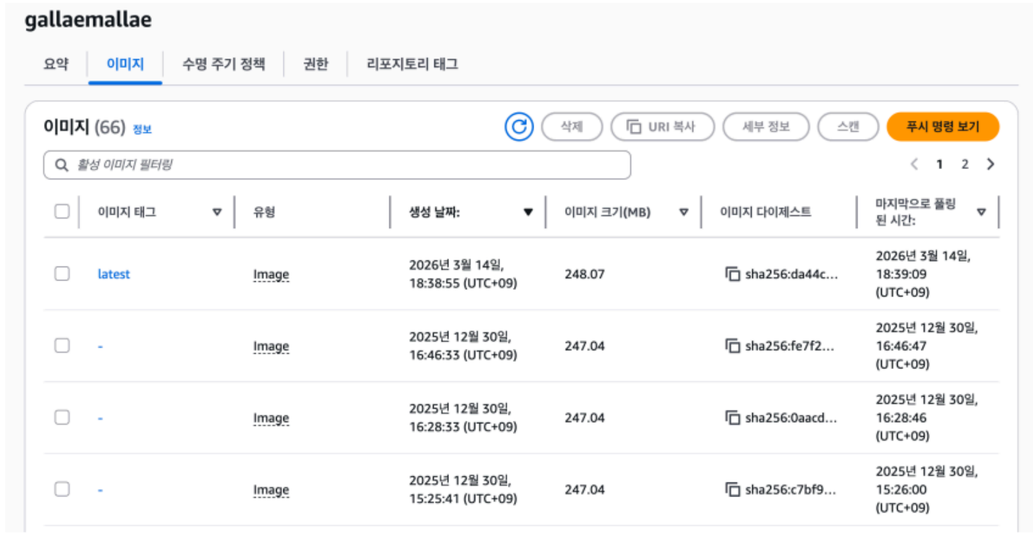
- 동시성 제어 적용 후 재발급 API를 20건 동시 요청 테스트 시 Race Condition 없이 정상 처리됨을 확인

4. 안정적이고 일관된 배포를 위한 AWS 활용 CI/CD 파이프라인 구축

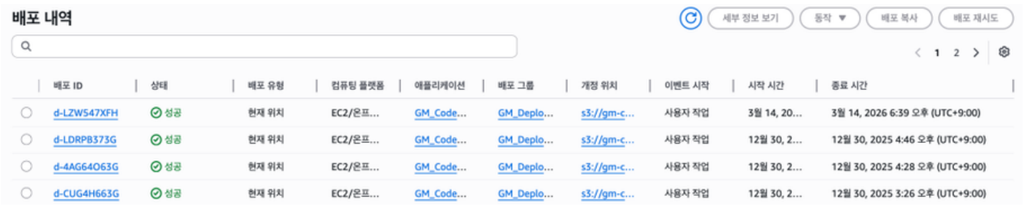
기술스택 : AWS, Docker, GitHub Actions



[AWS ECR] Docker image 중앙 관리



[AWS CodeDeploy] 배포 절차 표준화 및 중앙 관리



[도입 배경]

- 안정적이고 일관된 배포를 위한 표준화 및 자동화 환경 구축

[주요 작업]

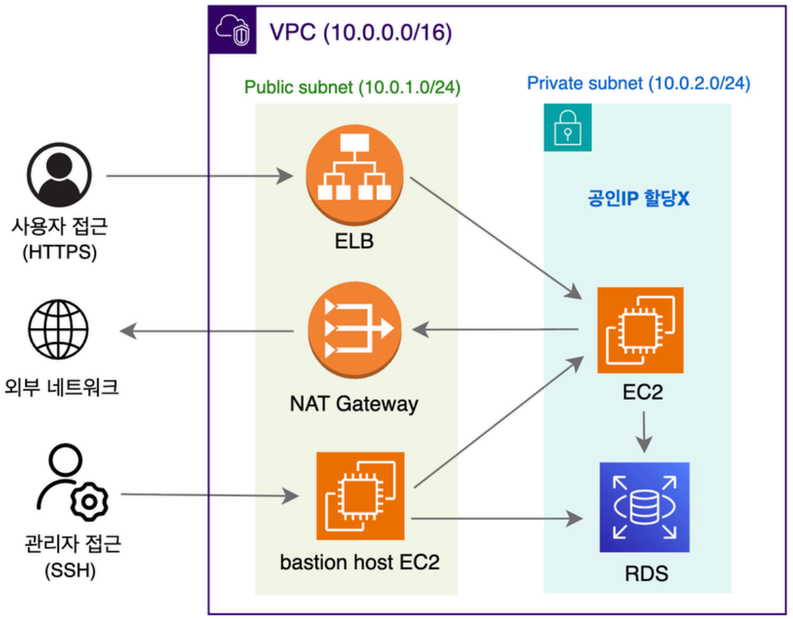
- Docker, GitHub Actions, AWS 를 활용한 CI/CD 파이프라인 구축

[성과]

- Docker image 중앙 관리와 표준화된 배포 절차를 통해 안정적이고 일관된 CI/CD 환경 구축

5. 보안성을 높인 AWS 가상 사설 클라우드 환경 구축

기술스택 : AWS



Public subnet route table

Destination	Route
내부 사설 ip	local
외부 공인 ip	Internet Gateway

Private subnet route table

Destination	Route
내부 사설 ip	local
외부 공인 ip	NAT Gateway

[도입 배경]

- 접근 통제가 부족한 구조로는 보안성을 확보하기 어렵다고 판단하여 서브넷 기반의 보안형 인프라 환경을 구축

[주요 작업]

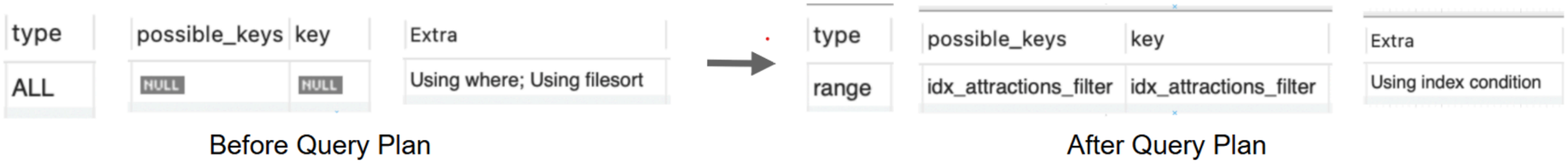
- 접근 경로 통제 및 목적에 맞는 접근만 허용하는 VPC 환경 구축
- 각 리소스의 보안 그룹과 각 서브넷의 라우팅 테이블을 최소 권한의 원칙에 맞게 설정
- Route 53으로 도메인을 연결하고, ELB(ALB)에 TLS 인증서를 적용해 HTTPS 기반의 암호화 통신 환경을 구축

[성과]

- 서브넷 분리와 접근 경로 통제를 통해 내부 자원의 직접 노출을 차단하여 보안성을 강화
- 관리자, 사용자, 서버의 통신 경로를 분리해 목적에 맞는 접근만 허용함으로써 불필요한 네트워크 접근을 차단하고 접근 통제 수준 향상

6. 조건 검색 성능 저하를 쿼리플랜 분석을 통한 인덱싱으로 최적화

기술스택 : Spring Boot, MySQL



[문제 및 원인]

- 카테고리, 등록일 범위, 좋아요 수 조건에 맞춰 등록순으로 관광지를 검색하는 API에서 성능 저하가 발생
- 쿼리 플랜을 분석한 결과 :
  - type = ALL로 약 23만 건의 테이블을 대상으로 Full Table Scan을 수행하여 조건과 무관한 데이터까지 광범위하게 조회
  - 검색 조건에 활용할 수 있는 인덱스가 없어 B+Tree 기반으로 탐색 범위를 줄이지 못함
  - Using filesort가 발생해 ORDER BY created_at 처리 시 별도의 추가 정렬 연산 수행

Query

```
SELECT a.attr_id, a.title …
FROM attractions a
WHERE a.created_at >= ?
AND a.content_type_id = ?
AND a.like_count >= ?
ORDER BY a.created_at
LIMIT ?, ?
```

[해결 과정]

- 조회 조건과 정렬 기준으로 사용되는 created_at, like_count, content_type_id를 대상으로 복합 인덱스 구성
  - 조회 비중이 높고 변경 빈도가 낮은 관광지 데이터 특성을 고려하여 인덱스를 적용해도 된다고 판단
- created_at을 인덱스 선두 컬럼으로 배치하여
  - B+Tree에서 등록일 조건에 맞는 시작 위치를 찾아 필요한 범위만 순차 탐색
  - 인덱스 자체가 created_at 순으로 정렬되어 있어 ORDER BY created_at의 추가 정렬 제거

[성과]

- Full Table Scan → Index Range Scan
- Using filesort 제거
- DB 조회시간 약 95% 단축